

A vulnerability detection algorithm based on residual graph attention networks for source code imbalance (RGAN)

Mingwei Tang^{a,*}, Wei Tang^a, Qingchi Gui^a, Jie Hu^b, Mingfeng Zhao^c

^a School of Computer and Software Engineering, Xihua University, Chengdu 610039, Sichuan Province, China

^b School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu, 611756, Sichuan Province, China

^c China Mobile Group Design Institute Co., Ltd. Sichuan Branch, Chengdu 610045, Sichuan Province, China

ARTICLE INFO

Keywords:

Source code vulnerability detection
Sequence neural network
Graph neural network
Attention mechanism
Imbalance processing

ABSTRACT

It is essential to detect potential vulnerabilities in software to ensure its safety. As software systems become more complex, traditional static vulnerability detection methods perform poorly. Currently, deep learning-based vulnerability detection models only extract source code vulnerability features using sequences or graphs. Sequential neural networks ignore structural information in the code, such as control flow diagrams and data flow diagrams. Additionally, graph neural networks cannot accurately extract features due to the lack of effective methods for extracting nodes' features and aggregating global information. To address the above issue, we propose a vulnerability detection algorithm based on residual graph attention networks for source code imbalance (RGAN). Firstly, a local feature extraction module (PE-BL-A module) is designed. Using the sequence neural network, the module extracts various useful features, including node features in a control flow diagram based on local semantic features. Secondly, we present the Residual Graph Attention Network module (RGAT). To learn and update node features along the control flow direction, the module uses a graph attention network with residual connections. In this module, a mean biaffine attention pooling mechanism is proposed that can extract total graph vulnerability features more effectively. Thirdly, a dynamic cross-entropy loss function is designed. Using this function, it can handle sample imbalances during training. Finally, experiments conducted on several benchmark datasets demonstrate that the proposed model achieves state-of-the-art results.

1. Introduction

With the rapid development of computer systems and internet technologies, more and more software systems have emerged at an astonishing rate (Cui, Zong, Xie, & Tang, 2023). These software systems permeate all aspects of our lives, from everyday activities such as eating, shopping, and transportation, to various industrial controls, mechanical manufacturing, logistics, and even modern military construction and national defense security. With the increasing number of functions and complexity of these software systems, they inevitably contain various vulnerabilities. These vulnerabilities are a result of unreasonable development specifications or oversights in the software development process. Once these vulnerabilities are discovered and exploited by malicious attackers, they can cause significant losses to individuals, society, and the economy.

Source code security incidents have become increasingly frequent in recent years. Based on the statistical data from the United States

National Vulnerability Database (NVD)¹ (American Information Technology Laboratory, 2023) shown in Fig. 1, the number of vulnerabilities has been on the rise since 1997. There has been a dramatic increase in vulnerabilities since 2017. In 2017, the hacker group Shadow Brokers disclosed a serious Windows vulnerability called Eternal Blue, which was leaked from the NSA's arsenal. The number of cases reported in 2022 alone was over 25,000. Among them, 11,225 vulnerabilities were rated as medium-to-high severity, accounting for 44.71% of the total vulnerabilities in 2022. A total of over 200,000 vulnerabilities have been reported to the NVD, which has resulted in significant losses for society.

Using this vulnerability, hackers produced WannaCry, a notorious ransomware that affected many countries, including governments, banks, universities, electric power generation, communications, energy, airports, and other industries and important infrastructures, causing serious damage. Due to vulnerabilities in its security system, Facebook's world-renowned social networking site was hacked in 2018, resulting in

* Corresponding author.

E-mail addresses: Tangwei@mail.xhu.edu.cn (M. Tang), 1506499935@qq.com (W. Tang), guiqingci@mail.xhu.edu.cn (Q. Gui), jiehu@swjtu.edu.cn (J. Hu), zhaomingfeng@cndi.chinamobile.com (M. Zhao).

¹ <https://nvd.nist.gov/>

<https://doi.org/10.1016/j.eswa.2023.122216>

Received 3 May 2023; Received in revised form 16 October 2023; Accepted 16 October 2023

Available online 19 October 2023

0957-4174/© 2023 Elsevier Ltd. All rights reserved.

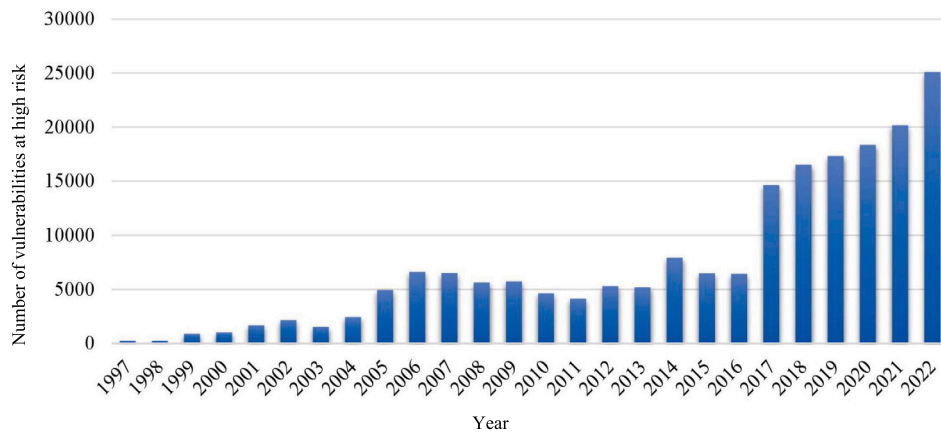


Fig. 1. The statistical data from the United States National Vulnerability Database.

the leak of nearly 30 million user records, including sensitive information such as names, contact information, and search histories (Security Guest, 2023). Facebook also suffered an information leak over the same time period due to software vulnerabilities, resulting in the leak of nearly 7 million private photos. In 2021, the well-known Java program log monitoring component Apache Log4j2 was vulnerable to a remote command execution vulnerability, affecting 60,644 open source packages involving 321,094 packages of related versions (FreeBuf Network Security Industry Portal, 2023).

Over the past several years, well-known manufacturers like Microsoft and Google have repeatedly revealed vulnerabilities in windows operating systems, Office packages, and numerous other software applications. It is possible for attackers to exploit these vulnerabilities to steal user information, attack governments, banks, and other important industries and infrastructure, causing significant damage to society and the economy as a whole. Considering the high incidence of vulnerabilities, a quick and efficient automated source code detection solution is necessary to identify the risks of vulnerabilities in the source code.

During the software development process, a number of algorithms have been proposed for checking source code for vulnerabilities. Static code analysis is a commonly used method for detecting potential vulnerabilities, thereby reducing software maintenance costs and improving software quality. However, there are some limitations to traditional static analysis tools as outlined below. Static analysis tools traditionally require security experts to manually formulate vulnerability matching rules. In traditional static analysis tools, experts must formulate vulnerability matching rules manually, which requires a lot of time and resources. These rules are usually based on their knowledge and experience, so the performance and accuracy of the tools will be limited by their capabilities. It can be difficult to describe complex vulnerabilities simply with rules, which means more time and resources are needed for research and analysis. There can be no vulnerability that is difficult to describe. Several vulnerabilities may be difficult to define with rules. For example, some vulnerabilities may involve interactions between multiple code files, multiple functions, and multiple modules. Since static analysis rules are typically designed for a single file or function, traditional static analysis tools are difficult to detect these vulnerabilities.

Machine learning and deep learning have been applied to a variety of fields over the past few years due to the rapid development of this technology. It has made significant advances in many fields such as natural language processing, image processing, speech recognition, and intelligent driving (Li, Tarlow, Brockschmidt, & Zemel, 2016). It is also widely used to detect source code vulnerabilities using deep learning methods. Researchers are developing new static analysis tools using machine learning and artificial intelligence techniques to overcome these limitations. In addition to improving the accuracy and efficiency

of vulnerability detection, these tools can automatically identify potential vulnerabilities by learning from a large number of code samples. So, detecting and analyzing source code automatically has become a major research area in software engineering in recent years.

Since source code and natural language are to some extent similar, some researchers have tried to directly migrate natural language processing technology to source code. Researchers have achieved very promising results on multiple source code processing tasks, such as code generation (Ahmad, Chakraborty, Ray, & Chang, 2021; Guo et al., 2020; Khoshnam & Baraani-Dastjerdi, 2022; Wang, Wang, Mi, et al., 2021), code summary generation (Feng et al., 2020), code migration (Yu, Bohnet, & Poesio, 2020), clone detection (Yamaguchi, Schmidt, & Effendi, 2014; Yu, Bohnet, & Poesio, 2020), and vulnerability detection (Fu & Tantithamthavorn, 2022). In some methods, source code sequences are treated as text sequences and classified using text classifications. According to Zhao et al. ProAttack, based on the prompt, is an efficient and novel method for performing clean-label backdoor attacks (Zhao, Wen, Tuan, Zhao, & Fu, 2023). In addition to not requiring external triggers, the method ensures proper labeling of poisoned samples, improving the stealthiness of the backdoor attack.

The model does not use structural information from the source code, so the features it extracts are not as accurate as they could be.

As a result, a graph neural network model and graph structured data are proposed for detecting source code vulnerabilities (Cao et al., 2022; Fu & Tantithamthavorn, 2022; Hin, Kan, Chen, & Babar, 2022). These approaches can be used to effectively utilize control flow diagrams and data flow diagrams in source code. It is common for graph neural network-based methods to not fully exploit the sequence information in nodes, which also affects the subsequent extraction of full-graph features.

These problems have been addressed by researchers through the development of some improved methods. A number of researchers have used node embedding techniques to embed sequence information into node features. A graph neural network can benefit from these embedding techniques by propagating and aggregating node features more effectively. The relationship information between nodes in a graph can also be used to extract features by some models based on graph convolutional networks (GCNs). Detection of source code vulnerabilities with these methods has shown good results.

There has been considerable progress in the application of pre-trained language models in the field of natural language processing, and these models have shown excellent results in a wide range of natural language tasks. BERT is one of the most famous and successful pre-trained language models (Devlin, Chang, Lee, & Toutanova, 2019). Through large-scale unsupervised corpus training, general language representations can be obtained that can then be fine-tuned for specific tasks in natural language processing. With the BERT model, meaning

and relationship between words can be better captured, resulting in a more semantic representation of words.

Since source code is similar to natural language to a certain extent, some researchers attempt to use pre-trained language models to process source code. Pre-trained language models can be applied to source code since it contains large amounts of text. In addition to code completion, code comment generation, code clone detection, code change suggestion, and code similarity measurement, these applications also include code clone detection and code change suggestion.

The use of pre-trained language models can improve the quality of code representation by capturing contextual and semantic information about code fragments. The pre-trained language model can also be used to generate code. For example, in code comment generation, it can automatically generate comments based on a given code fragment, thus improving readability and maintainability. In the field of natural language processing and source code analysis, pre-trained language models provide new ideas and methods (Guo et al., 2022; Hanif & Maffei, 2022; Liu et al., 2019; Wang, Wang, Joty, & Hoi, 2021).

Although researchers have proposed many new algorithms, deep learning-based vulnerability detection models have some limitations. It is typical for these models to only use a single sequence or graph feature extraction method to extract the vulnerability features of source code, ignoring structural information such as control flow chart and data flow diagram. Furthermore, sequential neural network-based models often fail to consider the code's inherent structural information, while graph neural network-based models lack effective node feature extraction methods. This results in an inaccurate feature extraction. In addition, vulnerability codes are frequently distributed unevenly, so vulnerability detection models need to handle imbalanced datasets as well. However, these vulnerability detection models cannot handle both sequence and graph information simultaneously, nor can they handle the imbalanced data problem. To address the above problems, this paper proposed a vulnerability detection algorithm based on residual graph attention networks for source code imbalance (RGAN). The RGAN model consists of four main components: local feature extraction (PE-BL-A module), node feature fusion (M-BFA module), residual graph attention network (RGAN) and multilayer perceptron module (MLP). In summary, the main contributions and innovative features are as follows:

- (1) A local feature extraction module (PE-BL-A) is designed, which is capable of extracting local feature representations of nodes.
- (2) A node feature fusion module is designed, which uses the M-BFA module to combine the local features of the nodes with the global features of the graph.
- (3) The residual graph attention network R-GAT was proposed to process the graph feature extraction and analysis.
- (4) A dynamic cross-entropy loss function is designed, which is able to handle the sample imbalance problem during training.

As a result, this paper is organized in the following manner. The second section introduces related work. The characteristics of the model are described in detail in Section 3. In Section 4, analyses and results of experiments are presented. Conclusions and directions for future work are presented in Section 5.

2. Related work

In this section, we will discuss some key technologies and algorithm theory related to this field. Firstly, some deep learning-based source code analysis algorithms are presented. Secondly, sequential neural network-based vulnerability detection approaches are described. Finally, graph neural networks are introduced as a vulnerability detection technique.

2.1. Deep learning-based source code analysis algorithm

Deep learning-based source code processing involves analyzing, understanding, and manipulating source code using deep learning techniques. It can be used for many software engineering tasks, including modeling, generating, optimizing, and detecting vulnerabilities. A variety of analyses and processing can be performed using deep learning technology to automatically extract crucial features from source code. Natural language and source code share many similarities. For instance, they both have grammatical structure, lexical features, and contextual semantics. Therefore, some natural language processing techniques can be adapted to source code processing tasks, such as detecting code defects, naming variables, and generating code.

Some research companies, such as IBM Research and Microsoft Research, have recently introduced the pre-trained language model BERT into source code processing. Several pre-trained language models have been developed using large-scale source code datasets, including CodeBERT (Feng et al., 2020) and C-BERT (Buratti et al., 2020). It can be applied to downstream tasks such as code translation, vulnerability detection, clone detection, code generation, etc. The OpenAI team also introduced the GPT3 (Brown et al., 2020) model to the code processing field. An evaluation index has been proposed for code generation, and a model for code generation has been trained using CodeX (Chen et al., 2021).

Additionally, Lacomis et al. proposed the DIRE model for restoring decompiled code to find variable names, as well as a cross-modal function-level binary code matching model to detect code similarity (Lacomis et al., 2019). The Coda deep learning model was developed by Fu et al. (2019) for replacing traditional decompilers for program decompilation. In addition, some researchers further enhanced the training process by adding a code de-obfuscation task. Feng et al. proposed a new model DOBF (Roziere, Lachaux, Szafraniec, & Lample, 2021), which can de-obfuscate and increase the readability of obfuscated code. Using DPCNN and different layers of LSTM (Hochreiter & Schmidhuber, 1997) networks, Yu, Zheng, et al. (2020) built a cross-modal function-level binary code matching model for code similarity detection. Source code processing has benefited greatly from these results, which have introduced new ideas and methods, and have improved efficiency and accuracy.

2.2. Sequential neural network vulnerability detection method

The sequential neural network is a neural network specially designed for processing sequence data, such as text, speech, time series, etc. Vulnerability detection can be automated using sequential neural networks by learning patterns from source code. To detect potential vulnerabilities, a recurrent neural network (RNN) or a long-short-term memory network (LSTM) can be used to learn control flow in code.

A CNN network was used by Russell et al. (2018) to extract code features for vulnerability detection using source code as a sequence of natural language texts embedded with Word2Vec technology (Mikolov, Sutskever, Chen, Corrado, & Dean, 2013). Using a multilayer bidirectional LSTM neural network, Li et al. (2018) extracted multiple code fragments involved in the parameters of the focus, and used a multilayered bidirectional LSTM neural network to detect vulnerabilities. Li et al. (2016) added edge prediction and node alignment tasks to the BERT model. In this study, a cross-modal programming language pre-training language model is proposed using programming languages such as Python, Java, and PHP. Using the trained model for downstream tasks, such as detecting source code vulnerabilities, CodeBERT has achieved significant performance gains (Feng et al., 2020).

As a result, these neural network-based vulnerability detection models cannot provide structural information, although structural information is crucial for static analysis. The model cannot determine the control transfer relationship within a program without control flow

information. Value flow relationships within a program cannot be perceived without data flow information. The lack of such information makes the overall model's performance less powerful than expected. There are sequential neural network models like GraphCodeBERT (Guo et al., 2020) that integrate graph structural information into the sequence model, trying to make the model learn the structural information of the code in the sequence. However, the improvement is not obvious enough.

Since the graph structure of the code cannot be directly perceived by the sequence model, the vulnerability detection model based on sequence neural networks suffers from lack of structured information. Some researchers have investigated how graph neural networks can be used to detect source code vulnerabilities in order to solve this problem. In order to improve the performance of the model, they use the control flow and data flow graph of the code to represent the structural information of the program.

2.3. Graph neural network vulnerability detection method

A graph neural network is a type of neural network that analyses graph data and can learn the topology of graphs and the relationships between nodes. It is possible to use graph neural networks to detect vulnerabilities in source code by representing control flow graphs or data flow graphs. The graph convolutional network (GCN) can be used, for instance, to detect potential vulnerabilities in a control flow graph.

In the field of source code processing, graph neural networks are an important technology precisely because sequential neural networks are incapable of using structured information from codes. Graph neural networks combine the features of nodes in a graph structure and the relationships between them. They perform feature extraction and representation learning on the entire graph.

In source code processing, graph structures can be control flow diagrams, data flow diagrams, dependency graphs, etc. By using a graph structure, graph neural networks can take full advantage of the relationships between nodes. In this way, it will be better able to mine the structured information in the code and detect vulnerabilities more accurately. A number of graph neural networks, such as the graph convolutional network (GCN) and the graph attention network (GAT), have been used in recent years to detect vulnerabilities in source code. By learning the node information in the graph structure, these models extract features and detect vulnerabilities in the code.

For the detection of function-level source code vulnerabilities, Zhou, Liu, Siow, Du, and Liu (2019) used a gated graph neural network. To embed the code blocks of the nodes, Devign uses Word2Vec to embed the code attribute graph processed by Joern (Yamaguchi et al., 2014). To obtain the full graph feature representation, Zhou et al. also proposed a convolution-based node information fusion method in one dimension. Using gated graph neural networks (Li et al., 2016), Chakraborty, Krishna, Ding, and Ray (2022) created the ReVeal model. As well as code property graphs, it uses multiple datasets to evaluate the model.

It also evaluates the benefits and disadvantages of vulnerability detection datasets constructed differently, and proposes the REVEAL unbalanced real datasets. Using the sequence features of the function, Hin et al. (2022) proposed the LineVD model, which learns program dependencies, and detects vulnerability code at the sentence level. ReGVD has been proposed by Nguyen et al. (2022) for function-level vulnerability detection. Although it also uses graph neural networks for learning, it does not use any structured information that comes with the source code. Graph neural networks were utilized to extract features from code token sequences using ReGVD's floating window mechanism. Xiao et al. constructed the input subgraph by extracting forward-backward relationships between the points of interest in the program dependency graph (Cheng, Wang, Hua, Xu, & Sui, 2021). During the learning process of the TopK method, the graph neural network model is pruned according to the JK-Net style, which reduces resource consumption and ensures accuracy. Most researchers studying graph neural networks for vulnerability detection focus on designing models and building graph data, but few study node feature extraction.

3. Methodology

3.1. Definition of the problem

Based on binary classification tasks on graphs, we define the task of detecting function-level source code vulnerabilities in this paper. Therefore, it is possible to determine whether or not a program contains vulnerabilities. All source code text collections are defined as $\{(c_i, y_i) \mid c_i \in \mathbb{C}, y_i \in \mathbb{Y}\}_{i=1}^N$ in this paper, where $\mathbb{C}$ represents all source code text collections, and $\mathbb{Y} = \{0, 1\}$ represents label collections. 0 represents common code samples, and 1 represents vulnerability code samples. The control flow graph of source code is used as the model input for classifying source code vulnerabilities.

A control flow diagram $cf_g(\mathcal{V}, X, A) \in \mathcal{G}$ is designed for any source code sample c_i , where $\mathcal{V}$ represents an array of n nodes and $X \in \mathbb{R}^{n \times d}$ is the feature matrix of the nodes. Each node $v_i \in \mathcal{V}$ is represented by a dimensional vector $x \in \mathbb{R}^d$. $A \in \{0, 1\}^{n \times n}$ stands for adjacency matrix. As long as $A_{i,j} = 1$, there is an edge between node i and node j . The function maps $f: \mathcal{G} \rightarrow \mathbb{Y}$ is used to determine whether a given source code sample is vulnerable. In order to learn this mapping function f , it is necessary to minimize a loss function that includes a regularization term, as shown in Eq. (1):

$$\min \sum \mathcal{L}(f(c_f g_i(\mathcal{V}, X, A), y_i \mid c_i)) + \lambda \|\theta\|_2^2 \quad (1)$$

where $\mathcal{L}(\cdot)$ is the adjustable weight, and λ is the cross-entropy loss function. There is an imbalance in the distribution of vulnerability detection samples, with a far greater number of ordinary samples than vulnerability samples in actual scenarios. In order to solve this problem, the cross-entropy loss function is designed. A dynamic weight cross-entropy loss is proposed, as shown in Eq. (2).

$$\min \frac{1}{N} \sum_i^N - [\exp(w_i) \cdot y_i \cdot \log(p_i) + \exp(1 - w_i) \cdot (1 - y_i) \cdot \log(1 - p_i)] \quad (2)$$

where w_i is the category weight in dynamic training, which is dynamically updated according to the proportion of samples. There is a value of 0 or 1 associated with sample i label y_i . p_i is the probability of classifying sample i as positive.

3.2. Residual graph attention network model (RGAN)

The RGAN model consists of four main components: local feature extraction (PE-BL-A module), node feature fusion (M-BFA module), residual graph attention network (RGAN) and multilayer perceptron module (MLP), as shown in Fig. 2.

- (1) PE-BL-A module: It is used to extract and represent local features of nodes. An attention mechanism is used to fuse information between different features in this module.
- (2) M-BFA module: The M-BFA Module is used to fuse the features of nodes. Using the attention mechanism, this module fuses multiple features of nodes to provide a more comprehensive representation of nodes.
- (3) Residual graph attention module: A graph's global features are extracted using the Module. The R-GAT residual graph attention network is used for graph convolution, and an attention mechanism between feature maps is used for information fusion.
- (4) MLP module: Graphs are classified using machine learning by the MLP Module. Multilayer perceptrons (MLPs) and softmax activation functions are used to calculate probability.

As a solution to the imbalance problem in the training data, RGAN uses the dynamic cross-entropy loss function instead of the traditional cross-entropy loss function. Dynamic cross-entropy loss functions can effectively address class imbalance, which facilitates better adaptation to class differences.

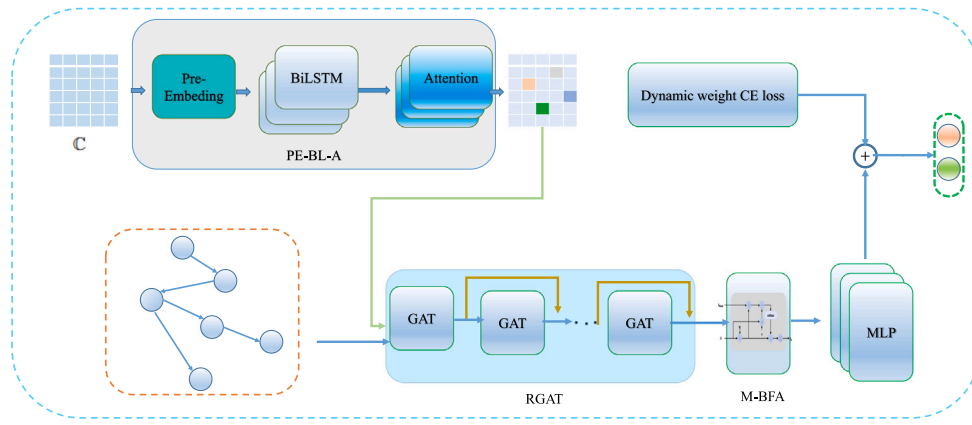


Fig. 2. Residual Graph Attention Network Model (RGAN).

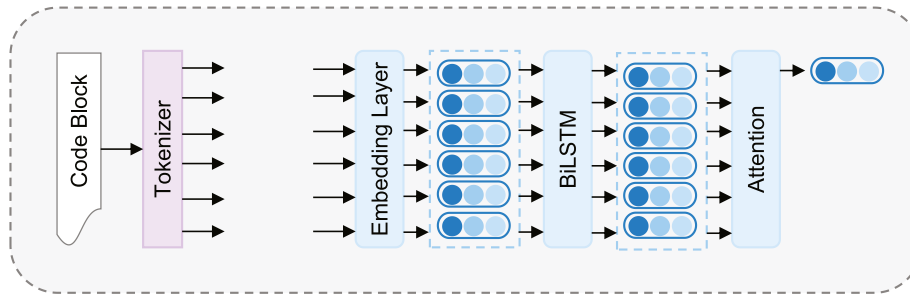


Fig. 3. PE-BL-A module.

3.3. Local feature extraction module based on attention mechanism (PE-BL-A module)

A graph-structured dataset is used as the input data type in this paper. In the preprocessing process, the Joern tool is used to extract the corresponding control flow chart from the function-level source code. The control flow chart is compressed by line, so that a line of code corresponds to a control flow chart node. This paper proposes the PE-BL-A module, which is used to extract textual features from local code blocks as node representations, as shown in Fig. 3. The first step is to segment the local code block using the pre-trained BPE tokenizer, as shown in Eq. (3).

$$\text{Tokenizer (code block)} \rightarrow [CLS], \text{Token 1, Token 2, } \dots, \text{TokenN, [SEP]} \quad (3)$$

In the second step, we will use the embedding layer to convert the token sequence after word segmentation into a corresponding vector representation, and then get the feature vector matrix $X = \{x_1, x_2, \dots, x_n\}$ of the local code block, as shown in Eq. (4).

$$X = \text{EmbeddingLayer}(\text{token_id_list}) \quad (4)$$

In the following step, the local code block feature vector matrix is input to the bidirectional LSTM layer, which updates the vector representation of each token using the context semantic learning ability to obtain a new hidden layer state vector matrix H , as shown in Eq. (9).

$$H_t = BiLSTM(X) \quad (5)$$

$$H_t = [H_l; H_r] \quad (6)$$

$$H_l = \{\bar{h}_1, \bar{h}_2, \dots, \bar{h}_n\} \quad (7)$$

$$H_r = \{\vec{h}_1, \vec{h}_2, \dots, \vec{h}_n\} \quad (8)$$

$$H = H_l + H_r \quad (9)$$

Finally, the self-attention mechanism is used to aggregate the state vector matrix H in the hidden layer, so that certain important features are emphasized by the model. According to its calculation, it is as follows:

$$q = h_{[CLS]} \cdot \omega, K = H \cdot W^K, V = H \cdot W^V \quad (10)$$

$$\text{Attention}(q, K, V) = \text{softmax} \left(\frac{q \cdot K^T}{\sqrt{d_k}} \right) \cdot V \quad (11)$$

$$h_v = \text{Attention}(q, K, V) \quad (12)$$

where ω is the learnable vector. W^K and W^V are the learnable matrices. h_v represents the final node feature representation, as shown in Eq. (12).

3.4. Mean biaffine attention pooling module (M-BFA)

Graph neural networks are learned and updated by averaging the final states of all nodes to obtain the final representation of the whole graph to be classified. In Fig. 4, the M-BFA readout layer is proposed for node information aggregation in the RGAN model. Biaffine attention and global attention pooling are incorporated into the mean biaffine attention layer (Yu, Bohnet, & Poesio, 2020).

In order to fusion global graph information, supernode v_s is assumed to play a leading role. Next, the attention score is calculated between each node and its supernode. Lastly, the full-graph feature representation is calculated using the weighted aggregation of all nodes. Experimental results show that the best model effect can be achieved by using the mean value of all nodes as the vector representation of

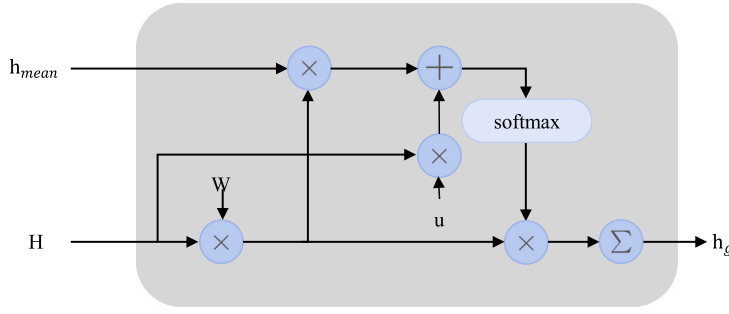


Fig. 4. The mean biaffine attention pooling mechanism (M-BFA).

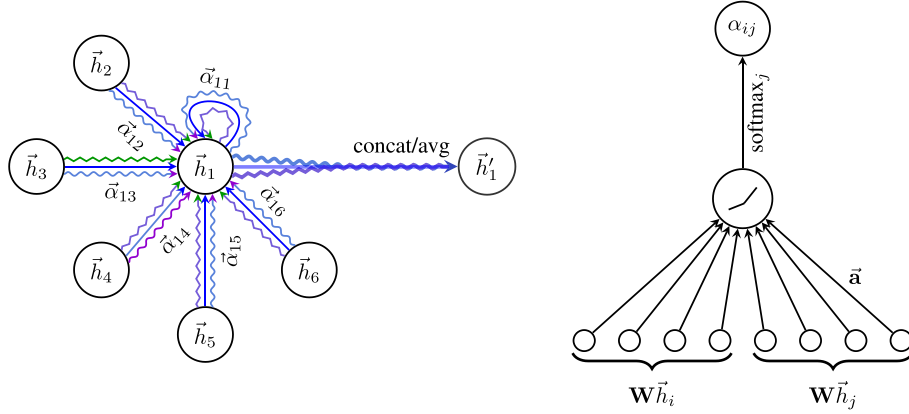


Fig. 5. Graph Attention Network (GAT).

super nodes. According to Fig. 4, M-BFA can be calculated using the following formula.

$$h_{\text{mean}} = \frac{\sum_{i=1}^n h_i}{n} \quad (13)$$

$$h_{fi} = h_i^T \cdot W \quad (14)$$

$$e_i = h_{fi}^T \cdot h_{\text{mean}} + h_i^T \cdot u \quad (15)$$

$$a_i = \frac{\exp(e_i)}{\sum_{j=0}^n \exp(e_j)} \quad (16)$$

$$h_g = \sum_{i=0}^n a_i \cdot h_{fi} \quad (17)$$

where W is a learnable weight matrix, u is a learnable weight vector, h_i is the final hidden state of node v_i , and h_g indicates the vector presentation of the graph, as shown in Eqs. (16)–(17). Specially, we choose the mean h_{mean} of all node' hidden representations as the hidden representation of supernode v_s .

3.5. Residual graph attention neural network RGAT

There is a spatial graph neural network called Graph Attention Network (GAT) (Veličković et al., 2017). Instead of using the Laplacian matrix as the weight of aggregating neighbor node information in GCN, the attention mechanism is used to learn an attention weight, as shown in Fig. 5.

The RGCN method extracts code features from control flow diagrams for vulnerability classification using control flow diagrams as input. Graph attention neural networks with residual connections are constructed to better utilize graph structure information. In graph attention neural networks, the filter matrix for information dissemination

is not calculated using Laplacian regularization, but rather by calculating the attention weight between the current node and neighbors. To calculate the output feature vector of each node in the graph attention neural network, the node feature vectors of its neighbors are combined. To improve information transfer, this method also uses residual connections. A multi-head attention mechanism allows the network to learn different attention weights simultaneously for better feature extraction in different feature spaces. Graph structure can be used to propagate information and extract features via the weighted summation of neighboring nodes, as shown in Eq. (18).

$$\alpha_{ij} = \frac{\exp\left(\text{LeakyReLU}\left(\vec{a}^T \left[W\vec{h}_i \parallel W\vec{h}_j\right]\right)\right)}{\sum_{k \in \mathcal{N}^i} \exp\left(\text{LeakyReLU}\left(\vec{a}^T \left[W\vec{h}_i \parallel W\vec{h}_k\right]\right)\right)} \quad (18)$$

When aggregating neighbor nodes, the adjacency matrix also aggregates its own information since it contains self-connections. It is calculated as follows when multi-head attention is used, as shown in Eq. (19).

$$\vec{h}'_i = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in \mathcal{N}_i} \alpha_{ij}^k W^k \vec{h}_j \right) \quad (19)$$

It is possible to avoid losing feature information when the number of layers increases by adding a residual connection between layers:

$$H^{(l+1)} = \text{GAT}\left(H^{(l)}, A\right) + H^{(l)} \quad (20)$$

where the current layer number is l , and the adjacency matrix is A , as shown in Eq. (20).

3.6. Module for unbalanced sample processing

In the training process, weighted random sampling is used to control the number of samples and maintain the balance of samples by assigning different weight parameters to samples of different categories.

The common formula for calculating sample weights is as follows, where $w_{y \in \{0,1\}}$ represents the weight of different categories and $N_{y \in \{0,1\}}$ represents the total number of samples of different categories, as shown in Eq. (21).

$$w_{y \in \{0,1\}} = \frac{1}{N_{y \in \{0,1\}}} \quad (21)$$

Dynamic weighted cross-entropy loss refers to the updating of category weights during the training process based on the number of positive and negative samples in each batch. Dynamic weighting allows for more effective training by adjusting the weight of each class dynamically based on the current batch. The equation for calculating the weights is as shown in Eq. (22).

$$w_{b0} = \frac{N_{b1}}{N_b}, w_{b1} = \frac{N_{b0}}{N_b} \quad (22)$$

There are a total of N_b samples in a batch. Whenever the number of samples of a class is 0, the calculation equation is shown in Eq. (23).

$$w_{b0} = \exp\left(\frac{N_{b1}}{N_b}\right), w_{b1} = \exp\left(\frac{N_{b0}}{N_b}\right) \quad (23)$$

3.7. Classifier learning

RGAN uses a multilayer perceptron (MLP) as the classification layer, which is the best common classifier. In this paper, our goal is to train a model that can learn graph representations from statement-level control flow graphs. A two-layer MLP can be constructed as shown in Eqs. (18) and (19) to predict whether a given function is vulnerable or not.

$$o = \text{softmax}(\sigma(U \cdot h_g) \cdot V) \quad (24)$$

$$y = \text{argmax}(o)$$

where U and V are two learnable weight matrix, $\sigma(\cdot)$ is an activation function, such as $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$, and $y \in \{0,1\}$ denotes the final predicted result, as shown in Eq. (24).

3.8. Complexity analysis

The whole model is based on GAT. the complexity of the model is also mainly determined by GAT. V is the number of vertices in the graph, E is the number of edges in the graph, F is the original feature dimension, and F' is the output feature dimension. The computational complexity is determined by the number of multiplications in the operation. The operation of GAT mainly involves the following two multiplication operations:

- (1) Feature mapping of vertices, W_{h_i} maps F -dimensional vectors to F' -dimensional space, W is a parameter matrix in $F \times F'$ dimensions, and the computational complexity of W_{h_i} is $O(F \times F')$. All vertices need to be mapped, the computational complexity is $O(V \times F \times F')$.
- (2) We are currently calculating the attention coefficients for the a() mapping. When a() maps a $2 \times F'$ -dimensional vector onto a real number, its computational complexity is $O(F')$. In the process of computing the attention coefficients, the graph has as many edges as there are edges (because each vertex computes the similarity coefficients with its neighboring vertices), then its computational complexity is $O(E \times F')$.

A subsequent session is primarily a weighted summation operation, and not a large number of high-complexity multiplications. By combining the above two links, the model first computes the feature mappings of all vertices and then computes the attention coefficients, so the computational complexity of model is $O(V \times F \times F' + E \times F')$.

Table 1
REVEAL dataset.

	Positive	Negative	Total
Train	1343	13 335	14 678
Valid	200	1667	1867
Test	185	1659	1844
Total	1728	16 661	18 389

4. Analyses and results of experiments

Compared to existing deep learning-based vulnerability detection methods, the neural network model proposed above is shown to be valid in this section. To ensure the integrity of the experiment, the selected comparative experimental model will be reproduced locally and various evaluation indicators will be compared and analyzed using the same data set and experimental environment. First, a brief introduction to the datasets is provided. Second, the experimental environment and parameters of the model are described, along with the evaluation indicators used. The experimental results are analyzed and presented in the final section.

4.1. Dataset

In this experiment, we train and validate our model using REVEAL, an unbalanced dataset drawn from the real world. There are 22,734 function-level code samples in the REVEAL dataset created by Chakraborty et al. (2022), of which 9.85% are vulnerability samples. To ensure a fair comparison, the REVEAL dataset has been divided by 8:1:1 and samples that cannot extract the control flow graph have been cleaned up. The training set contains 80% of the samples, the validation set contains 10% of the samples, and the test set contains 10% of the samples. As a general guideline, the sample sizes are in Table 1:

4.2. Environments for experimentation

Under the Linux operating system, this experiment was conducted. This paper proposes a vulnerability detection model in Python language, using PyTorch deep learning framework, PyTorch Lightning framework, and PyTorch Geometric graph neural network library. A detailed description of the parameter configuration of the software and hardware is shown in Table 2.

4.3. Hyperparameters

We use a unified setting for all of the experiments. Table 3 shows hyperparameters for our system. There are two GNN layers. There is one attention head. The batch size is 32. The dropout rate is 0.5. Adam is the optimizer. The learning rate is $5e-4$. The hidden layer size is 768. The epoch is 100.

4.4. Evaluation

This paper uses five basic evaluation indicators: Precision (P), Recall (R), $F1 - score$ ($F1$), false negative rate (FNR) and false positive rate (FPR). The definitions and formulas for each of these five indicators will be described in detail. TP (True Positive) is an indicator of the number of samples that were predicted to be vulnerable but also turned out to be vulnerable. The true negative (TN) is the number of samples predicted to be common samples that are actually common samples. FP stands for false positive, where the sample was predicted to be susceptible, but was actually susceptible. FN stands for false negative, where the sample is predicted to be common, but is actually exposed.

Table 2
Runtime environment.

	CPU Version	Intel(R) Core(TM) i9-10900X
Configuration of hardware	GPU Version	NVIDIA RTX 3090 24G
	Memory	64 GB
	Diskdrive	512G M.2 SSD
Configuration of software	System	Ubuntu 18.04.5 Server
	Python	3.7.6
	Pytorch	1.10.0
	Pytorch Lightning	1.8.3
	Pytorch Geometric	2.1.0

Table 3
Major hyperparameters for our model.

Defining parameters	Value
GNN layer	2
BiLSTM size	768
BiLSTM layer	1
BiLSTM dropout	0.2
Attention head	1
Batch size	32
Dropout rate	0.5
Optimizer	Adam
Learning rate	5e−4
Hidden layer size	768
Epoch	100

Precision (P) stands for precision rate, i.e., the percentage of samples predicted as vulnerability samples by the model that actually are vulnerability samples. The equation for its calculation is as follows:

$$Precision = \frac{TP}{TP + FP} \times 100\% \quad (25)$$

Recall (R): The proportion of samples that the model correctly predicts as vulnerability samples. The formula for its calculation is as follows:

$$Recall = \frac{TR}{TP + FN} \times 100\% \quad (26)$$

F1-score: Due to the balance between precision and recall, this is one of the most important evaluation indicators. The formula for its calculation is as follows:

$$F1 - score = \frac{2 \times P \times R}{P + R} \times 100\% \quad (27)$$

FNR (False Negative Rate): This is an indicator of the proportion of leaky samples used as normal samples during the modeling process. The formula for its calculation is as follows:

$$FNR = \frac{FN}{TP + FN} \times 100\% \quad (28)$$

FPR (False Positive Rate) is an indicator of how many samples are incorrectly predicted as vulnerability samples by the model when they are actually common samples. False positive rate: The formula for its calculation is as follows:

$$FPR = \frac{FP}{FP + TN} \times 100\% \quad (29)$$

It is important for a vulnerability detection model to have high precision, high recall, high F1-score, and low false positive and false negative rates.

4.5. Analyses and results of experiments

For several different questions, we evaluate and analyze several different models in this section and compare their performance with those in Table 4.

RQ1: Performance comparison of RGAN with existing models

There is a low recall rate and low F1-score in most comparison models due to sample imbalance, as shown in Table 5. A recall rate of 52.97% and an F1-score of 47.34% were achieved by RGCN. Moreover, the model presented in this section has a 47.02% false negative rate, which is highly advantageous.

Moreover, as shown in Table 6, this section also compares to the original REVEAL dataset proposal and its model ReVeal. In order to deal with dataset imbalances, the ReVeal model utilizes the SMOTE (Chawla, Bowyer, Hall, & Kegelmeyer, 2002) method. According to the Table 6, although the ReVeal model has a higher recall rate, the RGCN model has advantages in the other two categories.

RQ2: Ablation experiments

In order to better validate the proposed method, we performed ablation experiments on local feature extraction based on the attention mechanism, as well as dynamically weighted cross-entropy loss. The results of their experiments demonstrate that they are effective.

RQ2.1: The effectiveness of local feature extraction based on attention mechanisms

Based on the self-attention mechanism, this section explores the effectiveness of local feature extraction. In this chapter, the PE-BL-A module performs the corresponding feature fusion and confirms the effectiveness of the method proposed in this chapter. A comparison of the results for the two experiments can be found in Table 7. A conventional feature fusion method uses the mean, maximum, cumulative value of bidirectional LSTM or its final output as node features, as shown in Table 7.

Information fusion based on self-attention has better performance in multiple indicators than these other methods. As a result of the attention mechanism, important information can be more easily noticed in the local code sequence and noise can be reduced in the final feature representation. For example, if the attention mechanism works properly, it can detect more key information in the local code sequence and reduce the proportion of noise information in the final feature.

According to Table 7, the method using self-attention fusion achieved the highest F1-score of 47.34%, outperforming all comparison methods. Moreover, this section compares the performance of multi-head attention and self-attention. According to the experimental results, the multi-head attention mechanism does not improve self-attention performance.

In multi-head attention situations where there are two or four heads, self-attention mechanisms are the most successful, reaching a score of 47.34%, but they also have their own advantages and disadvantages. The self-attention mechanism is clearly superior when the number of attention heads is 8. In comparison with the PE-BL method above, the local feature extraction method in this chapter has a slightly higher recall rate, F1-score, and false negative rate.

Table 4

Compare models and the corresponding code links.

Model	URL
CodeBERT (Feng et al., 2020)	https://github.com/microsoft/CodeXGLUE/tree/main/Code-Code/Defect-detection
GraphCodeBERT (Guo et al., 2020)	https://github.com/microsoft/CodeXGLUE/tree/main/Code-Code/Defect-detection
CoText (Phan et al., 2021)	https://github.com/salesforce/CodeT5
CodeT5 (Wang, Wang, Joty, & Hoi, 2021)	https://github.com/salesforce/CodeT5
PLBART (Uddin Ahmad, Chakraborty, Ray, & Chang, 2021)	https://github.com/wasiahmad/PLBART
Russell et al. (2018)	https://github.com/VulDetProject/ReVeal
Devign (Zhou et al., 2019)	https://github.com/saikat107/Devign
ReVeal (Chakraborty et al., 2022)	https://github.com/VulDetProject/ReVeal
ReGVD (Nguyen et al., 2022)	https://github.com/daiquocnguyen/GNN-ReGVD

Table 5

Performance comparison of RGCN with existing models.

	Precision	Recall	F1-score	FPR	FNR
CodeBERT (Feng et al., 2020)	54.68	32.20	40.53	3.09	67.80
GraphCodeBERT (Guo et al., 2020)	71.83	21.61	33.22	1.03	78.39
CoText (Phan et al., 2021)	61.90	22.03	32.50	1.62	77.97
CodeT5 (Wang, Wang, Joty, & Hoi, 2021)	65.52	32.20	43.18	1.96	67.80
PLBART (Uddin Ahmad et al., 2021)	62.65	22.03	32.60	1.57	77.97
Russell et al. (2018)	37.04	13.51	19.80	2.70	86.49
Devign (Zhou et al., 2019)	28.99	25.00	26.85	6.99	75.00
ReGVD (Nguyen et al., 2022)	58.14	10.59	17.92	0.88	89.41
RGAN	42.79	52.97	47.34	7.96	47.02

Table 6

Performance comparison of RGCN and REVEAL models.

	Precision	Recall	F1-score
ReVeal (Chakraborty et al., 2022)	30.91	60.91	41.25
RGAN	42.79	52.97	47.34

RQ2.2: Dynamics-weighted cross-entropy loss's effectiveness

As shown in Table 8, unbalanced data negatively affects model performance, and dynamic weight cross-entropy loss is effective. Dynamic weight cross-entropy loss can improve the recall rate, F1-score, and false negative rate for models with imbalanced data. The use of R-GAT can further improve recall and F1-score without sacrificing accuracy and precision by replacing R-GCN with R-GAT. As a result, the RGCN model achieves the best performance on multiple evaluation metrics. In the RGCN model, the improvement is almost comprehensive, with precision rate of 42.79%, recall rate of 52.97%, F1-score of 47.34%, false positive rate of 7.96%, and false negative rate of 47.02%. It is proven that the method proposed in this chapter is valid.

5. Conclusions

Based on control flow charts and graph neural networks, this paper proposes a source code vulnerability detection approach RGAN to address the problem of frequent source code vulnerabilities in recent years. In addition to learning and detecting source code vulnerabilities, the model can also use text sequence information and intrinsic structured information. An unbalanced data set processing method is added to the RGAN model, which uses the sequence information from the source code and the graph structure information. Thus, the CSGVD model incorporates an attention mechanism to extract local code block features, replacing the residual graph convolutional network R-GCN with the residual graph attention network R-GAT. Additionally, the model proposes a dynamic weight cross-entropy loss function to replace the original cross-entropy loss function, which enhances the original model's performance on imbalanced datasets.

Source code processing based on deep learning, vulnerability detection based on sequence neural networks, and vulnerability detection based on graph neural networks are currently popular research directions. It is not possible to successfully detect every code sample in

the actual application process for various reasons. Based on function-level source code, this paper conducts binary classification vulnerability detection. This is essentially coarse-grained vulnerability detection, which cannot effectively pinpoint the vulnerability's location. Multi-feature integration will also be a significant development direction in software engineering in the future.

CRedit authorship contribution statement

Mingwei Tang: Conceptualization, Methodology, Writing – original draft. **Wei Tang:** Data curation, Methodology, Writing – original draft. **Qingchi Gui:** Visualization, Investigation. **Jie Hu:** Methodology, Writing – review & editing. **Mingfeng Zhao:** Writing – review.

Declaration of competing interest

We declare that we have no financial and personal relationships with other people or organizations that can inappropriately influence our work. There is no professional or other personal interest of any nature or kind in any product, service and company that could be construed as influencing the position presented in, or the review of, the manuscript entitled.

Data availability

The data that has been used is confidential.

Acknowledgments

This work is supported by the Scientific Research Funds project of Science and Technology Department of Sichuan Province, China (Nos. 2019YFG0508, 2019GFW131), Sichuan Key R&D project, China (No. 2023YFG0354), the National Natural Science Foundation of China (No. 61902324), Funds Project of Chengdu Science and Technology Bureau, China (Nos. 2017-RK00-00026-ZF, 2022-YF04-00065-JH, 2024**), Outstanding Youth Science Fund Project of the Sichuan Provincial Natural Science Foundation and Science, China (No. 2024**) and Technology Planning Project of Guizhou Province, China (No. QianKeHeJiChu-ZK[2021]YiBan319).

Table 7

The effectiveness of local feature extraction approach based on attention mechanism.

	Precision	Recall	F1-score	FPR	FNR
Mean	45.05	44.32	44.69	9.46	43.78
Max	40.15	56.22	46.85	5.00	68.11
Sum	45.05	44.32	44.69	6.39	56.64
PE-BL-A	46.63	41.08	43.68	5.24	58.92
MultiHeadAttention (2 Heads)	32.54	58.92	41.92	13.80	41.08
MultiHeadAttention (4 Heads)	34.88	61.08	44.40	12.72	38.92
MultiHeadAttention (8 Heads)	40.71	49.73	44.77	8.20	50.27
Self Attention	42.79	52.97	47.34	7.96	47.02

Table 8

Effectiveness of dynamic weighted cross-entropy loss.

	Precision	Recall	F1-score	FPR	FNR
CSGVD	51.14	24.32	32.97	2.59	75.68
CSGVD _{r-gat}	46.85	36.22	40.85	4.70	63.78
RGCN	39.42	44.32	41.73	7.66	55.68
CSGVD _{urs}	28.09	58.92	38.05	17.00	41.08
CSGVD _{r-gat+urs}	28.82	52.97	37.33	14.65	47.03
RGCN _{urs}	24.41	67.57	35.87	23.57	32.43
CSGVD _{fw}	27.03	59.46	37.16	18.08	40.54
CSGVD _{r-gat+fw}	28.54	64.32	39.53	17.96	35.68
RGCN _{fw}	35.97	49.19	41.55	9.89	50.81
CSGVD _{dw}	33.56	53.51	41.25	11.81	46.49
CSGVD _{r-gat+dw}	46.63	41.08	43.68	5.24	58.92
RGAN _{dw}	42.79	52.97	47.34	7.96	47.02

The random weight sampling method *urs*, cross-entropy loss *fw*, and dynamic weight cross-entropy loss *dw* are shown in Table 8.

References

- Ahmad, W. U., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Unified pre-training for program understanding and generation. In *NAACL-HLT 2021 - 2021 conference of the North American chapter of the Association for Computational Linguistics: Human language technologies, proceedings of the conference* (pp. 2655–2668).
- American Information Technology Laboratory (2023). National Vulnerability Database[EB/OL]. URL: <https://www.nist.gov/>.
- Brown, T., Mann, B., Ryder, N., Subbiah, N., Kaplan, J. D., Dhariwal, P., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901.
- Buratti, L., Pujar, S., Bornea, M., McCarley, S., Zheng, Y., Rossiello, G., et al. (2020). Exploring software naturalness through neural language models. arXiv preprint [arXiv:2006.12641](https://arxiv.org/abs/2006.12641).
- Cao, S., Sun, X., Bo, L., Wu, R., Li, B., & Tao, C. (2022). MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks. In *Proceedings - International conference on software engineering*, vol. 2022-May (pp. 1456–1468). <https://dx.doi.org/10.1145/3510003.3510219>.
- Chakraborty, S., Krishna, R., Ding, Y., & Ray, B. (2022). Deep learning based vulnerability detection: Are we there yet? *IEEE Transactions on Software Engineering*, 48(9), 3280–3296. <https://dx.doi.org/10.1109/TSE.2021.3087402>.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. d. O., Kaplan, J., et al. (2021). Evaluating large language models trained on code. arXiv preprint [arXiv:2107.03374](https://arxiv.org/abs/2107.03374).
- Cheng, X., Wang, H., Hua, J., Xu, G., & Sui, Y. (2021). DeepWukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Transactions on Software Engineering and Methodology*, 30(3), <https://dx.doi.org/10.1145/3436877>.
- Cui, J., Zong, L., Xie, J., & Tang, M. (2023). A novel multi-module integrated intrusion detection system for high-dimensional imbalanced data. *Applied Intelligence*, 53(1), 272–288. <https://dx.doi.org/10.1007/s10489-022-03361-2>.
- Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *Proceedings of the 2019 conference of the North American chapter of the Association for Computational Linguistics: Human language technologies* (pp. 4171–4186). Association for Computational Linguistics, <https://dx.doi.org/10.18653/v1/n19-1423>.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., et al. (2020). CodeBERT: A pre-trained model for programming and natural languages. In T. Cohn, Y. He, Y. Liu (Eds.), *Findings of ACL: EMNLP 2020, Findings of the Association for Computational Linguistics: EMNLP 2020* (pp. 1536–1547). Association for Computational Linguistics, <https://dx.doi.org/10.18653/v1/2020.findings-emnlp.139>.
- FreeBuf Network Security Industry Portal (2023). Zero to One takes you deeper into the log4j2 Jndi RCE CVE-2021-44228 vulnerability. URL: <https://www.freebuf.com/company-information/312180.html>.

- Fu, C., Chen, H., Liu, H., Chen, X., Tian, Y., Koushanfar, F., et al. (2019). Coda: An end-to-end neural program decompiler. *Advances in Neural Information Processing Systems*, 32.
- Fu, M., & Tantithamthavorn, C. (2022). LineVul: A transformer-based line-level vulnerability prediction. In *Proceedings - 2022 mining software repositories conference* (pp. 608–620). <http://dx.doi.org/10.1145/3524842.3528452>.
- Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., & Yin, J. (2022). UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the annual meeting of the Association for Computational Linguistics* (pp. 7212–7225).
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., et al. (2020). Graphcodebert: Pre-training code representations with data flow. arXiv preprint [arXiv:2009.08366](https://arxiv.org/abs/2009.08366).
- Hanif, H., & Maffei, S. (2022). VulBERTa: Simplified source code pre-training for vulnerability detection. In *Proceedings of the International Joint Conference on Neural Networks*, vol. 2022-July. <https://dx.doi.org/10.1109/IJCNN55064.2022.9892280>.
- Hin, D., Kan, A., Chen, H., & Babar, M. A. (2022). LineVD: statement-level vulnerability detection using graph neural networks. In *Proceedings of the 19th international conference on mining software repositories* (pp. 596–607).
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Khosnam, F., & Baraani-Dastjerdi, A. (2022). A dual framework for implicit and explicit emotion recognition: An ensemble of language models and computational linguistics. *Expert Systems with Applications*, 198, <https://dx.doi.org/10.1016/j.eswa.2022.116686>.
- Lacomis, J., Yin, P., Schwartz, E., Allamanis, M., Le Goues, C., Neubig, G., et al. (2019). Dire: A neural approach to decompiled identifier naming. In *2019 34th IEEE/ACM international conference on automated software engineering* (pp. 628–639). IEEE.
- Li, Y., Tarlow, D., Brockschmidt, M., & Zemel, R. S. (2016). Gated graph sequence neural networks. In Y. Bengio, & Y. LeCun (Eds.), *4th international conference on learning representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, conference track proceedings*. URL: <https://arxiv.org/abs/1511.05493>.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., et al. (2018). Vuldeepecker: A deep learning-based system for vulnerability detection. arXiv preprint [arXiv:1801.01681](https://arxiv.org/abs/1801.01681).
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., et al. (2019). RoBERTa: A robustly optimized BERT pretraining approach.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26.
- Nguyen, V.-A., Nguyen, D. Q., Nguyen, V., Le, T., Tran, Q. H., & Phung, D. (2022). ReGVD: Revisiting graph neural networks for vulnerability detection. In *Proceedings of the ACM/IEEE 44th international conference on software engineering: Companion proceedings* (pp. 178–182).
- Phan, L., Tran, H., Le, D., Nguyen, H., Annibal, J., Peltekian, A., et al. (2021). CoTextT: Multi-task learning with code-text transformer. In *Proceedings of the 1st workshop on natural language processing for programming* (pp. 40–47). Association for Computational Linguistics, <https://dx.doi.org/10.18653/v1/2021.nlp4prog-1.5>. Online. URL: <https://aclanthology.org/2021.nlp4prog-1.5>.
- Roziere, B., Lachaux, M.-A., Szafraniec, M., & Lampl, G. (2021). Dobf: A deobfuscation pre-training objective for programming languages. arXiv preprint [arXiv:2102.07492](https://arxiv.org/abs/2102.07492).
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., et al. (2018). Automated vulnerability detection in source code using deep representation learning. In *2018 17th IEEE international conference on machine learning and applications* (pp. 757–762). IEEE.
- Security Guest (2023). Compilation of domestic and international information leakage cases in 2018[EB/OL]. URL: <https://www.anquanke.com/post/id/169720>.
- Uddin Ahmad, W., Chakraborty, S., Ray, B., & Chang, K.-W. (2021). Unified pre-training for program understanding and generation. arXiv e-prints, arXiv:2103.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. arXiv preprint [arXiv:1710.10903](https://arxiv.org/abs/1710.10903).
- Wang, Y., Wang, W., Joty, S., & Hoi, S. C. (2021). CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 conference on empirical methods in natural language processing* (pp. 8696–8708). Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, <https://dx.doi.org/10.18653/v1/2021.emnlp-main.685>. URL: <https://aclanthology.org/2021.emnlp-main.685>.
- Wang, X., Wang, Y., Mi, F., Zhou, P., Wan, Y., Liu, X., et al. (2021). SynCoBERT: Syntax-guided multi-modal contrastive pre-training for code representation.

- Yamaguchi, F., Schmidt, N., & Effendi, D. B. (2014). Joern—the bug hunter’s workbench. Yu, J., Bohnet, B., & Poesio, M. (2020). Named entity recognition as dependency parsing. In *Proceedings of the annual meeting of the Association for Computational Linguistics* (pp. 6470–6476).
- Yu, Z., Zheng, W., Wang, J., Tang, Q., Nie, S., & Wu, S. (2020). Codecmr: Cross-modal retrieval for function-level binary source code matching. *Advances in Neural Information Processing Systems*, 33, 3872–3883.
- Zhao, S., Wen, J., Tuan, L. A., Zhao, J., & Fu, J. (2023). Prompt as triggers for backdoor attack: Examining the vulnerability in language models. <http://dx.doi.org/10.48550/arXiv.2305.01219>.
- Zhou, Y., Liu, S., Siow, J., Du, X., & Liu, Y. (2019). Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Advances in Neural Information Processing Systems*, 32.